Furkan Büyüksarıkulak

ID 22002097

EE-102, Section 003

Lab 5 : Seven Segment Display

Experiment 5

01.11.2023

AIM:

This lab project aimed to display hexadecimal number counter sequentially by using 7 segment display of Basys3 via VHDL. Moreover, we also learned what “persistence of vision” and “clock divider” are and how to implement.

DESIGN SPECIFICATIONS:

Our seven-segment display code designed in modular fashion, namely it consists one top module and 3 sub modules which are:

- Top module “main_7seg.vhd”
- Anode decoder “decoder_an.vhd”
- Counter “counter_7seg.vhd”
- LED driver “driver_LED.vhd”

Initially we have a top module, and it directly connects and combines the components we will mention below. The schematic of our seven segment dispaly module can be observed in (Figure1). The main module has 2 inputs and 2 outputs and these can be seen in (Table1).

IN	clk	1-bit	The 100Mhz clock from Basys3
IN	reset	1-bit	It resets the counting process by connecting to the counter component and receives switch input of Basys3.
OUT	anode	4-bit	It used as a conenction to the anodes of the Basys3 board and it is conencted to the “decoder_anod” module.
OUT	cathode	7-bit	Drives the LED’s

Table1 : main_7seg in and out ‘s

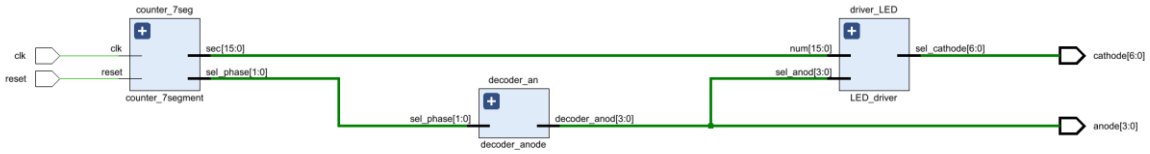


Figure1: Seven Segment Display Schematic

Our submodule anode decoder "decoder_an.vhd" decodes the 2 bit phase signal to 4 bit anode signal. For example, when the sel_phase is "00", the first segment will light up, the segment displays the first digit of the hex number according to the cathode combination. The schematic of anode decoder can be observed in (Figure2) . The module has 1 input and 1 output and these are can be seen in (Table2).

IN	sel_phase	2-bit	It takes the output signal of counter_7seg and this output is sel_phase
OUT	decoder_anod	4-bit	The resulting signal will be connected to the anode pins of Basys3.

Table2 : decoder_an in and out's

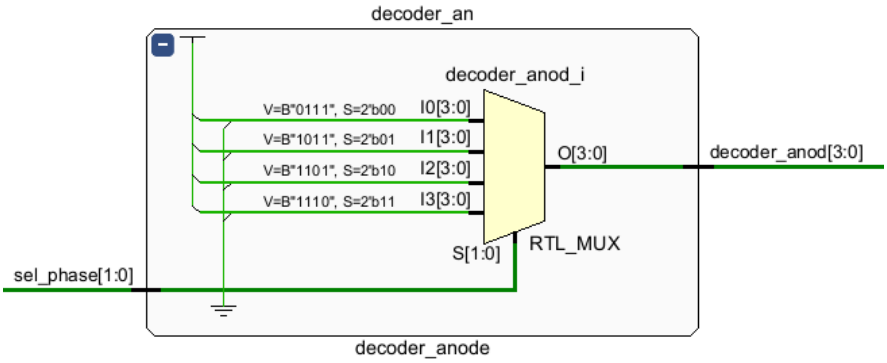


Figure2 : Anode Decoder Schematic

Our counter submodule "counter_7seg.vhd" is used for makes a clock. As we mentioned the Basys3 has 100Mhz clock internally and the counter submodule makes 1 Hz clock from this internal clock and determines the phase signal of the seven segment displays. The schematic of module can be seen in (Figure3) and the details of 2 input and 3 output in (Table3).

IN	reset	1-bit	It resets the counting process by connecting to the counter component and receives switch input of Basys3.
IN	clk	1-bit	The 100Mhz clock from Basys3
OUT	count_clck	28-bit	Counts the clock input
OUT	sel_phase	2-bit	Determine the phase of displays. Takes phase cycle to achieve persistence of vision.
OUT	sec	16-bit	Counts the seconds

Table3 : counter_7seg in and out's

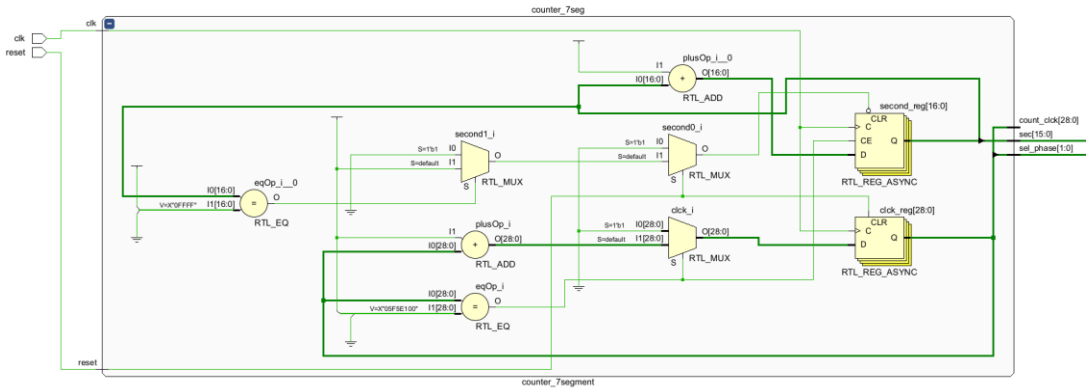


Figure3 : count_7seg Schematic

The submodule LED driver "driver_LED.vhd" was used to determine which LED would light up. It uses phase loop and 1Hz counter. Furthemore it selects the cathodes of seven segment display. This submodule has 2 input and 1 output and these are in (Table4) and the schematic of LED driver as in (Figure4).

IN	num	16-bit	Takes "sec" out of the "counter_7seg"
IN	sel_anod	4-bit	Takes the output of "decoder_an"
OUT	sel_cathode	7-bit	It drives LED's by using the hex cathode selector table (Table5).

Table4 : driver_LED in and out's

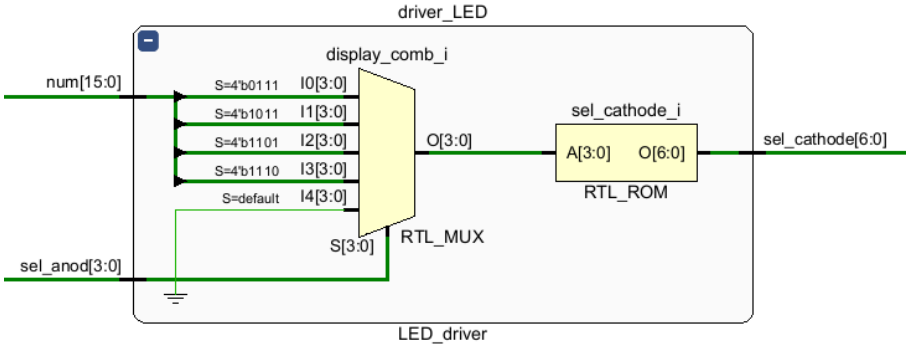


Figure4 : LED Driver Schematic

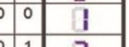
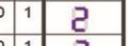
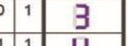
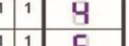
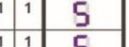
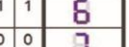
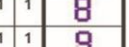
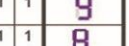
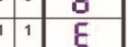
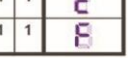
Digits	Input Lines				Output Lines							Display Pattern
	A	B	C	D	a	b	c	d	e	f	g	
0	0	0	0	0	1	1	1	1	1	1	0	
1	0	0	0	1	0	1	1	0	0	0	0	
2	0	0	1	0	1	1	0	1	1	0	1	
3	0	0	1	1	1	1	1	1	0	0	1	
4	0	1	0	0	0	1	1	0	0	1	1	
5	0	1	0	1	1	0	1	1	0	1	1	
6	0	1	1	0	1	0	1	1	1	1	1	
7	0	1	1	1	1	1	1	0	0	0	0	
8	1	0	0	0	1	1	1	1	1	1	1	
9	1	0	0	1	1	1	1	1	0	1	1	
A	0			0	0	1	1	0	0	1	1	
b	0	1	0	1	1	0	1	1	0	1	1	
C	0	1	1	0	1	0	1	1	1	1	1	
d	0	1	1	1	1	1	1	0	0	0	0	
E	1	0	0	0	1	1	1	1	1	1	1	
F	1	0	0	1	1	1	1	1	0	1	1	

Table 5: Hex cathode selector table

We have also two testbenches in our project. The first one is coded to simulates the top module and second one simulates the counter module.

METHODOLOGY:

We started the project by writing the submodule called "counter_7seg". Afterwards, we had to use a clock divider because Basys3 has an internal clock frequency of 100 Mhz and we need a 1Hz clock to count the seconds. When we examine the manual of Basys3, we can easily understand the working principle of the 7-segment display. There are 4 parts and there are 7 LEDs in each segment and they are controlled by the anode and cathode (Figure 5). As seen in the figure, there is a common anode for each part. This means that we cannot light up more than one display at the same time. To overcome this situation, we use "persistence of vision", which means that it switches between displays in a fast cycle and illuminates the corresponding number. This cycle is so fast that the human eye cannot catch it and it appears to be constantly illuminated. Our cycle duration is nearly 5.2ms. Then we designed the "driver_LED" code that would determine the corresponding digit to be displayed using the anode and cathodes. Finally we merged all submodules into top module "main_7seg". We wrote our testbench code to test the accuracy of

our code, and after observing from the simulation that we were successful, we wrote a constaint file “cons_7seg.xdc” to connect our inputs and outputs to the relevant pins on Basys3.

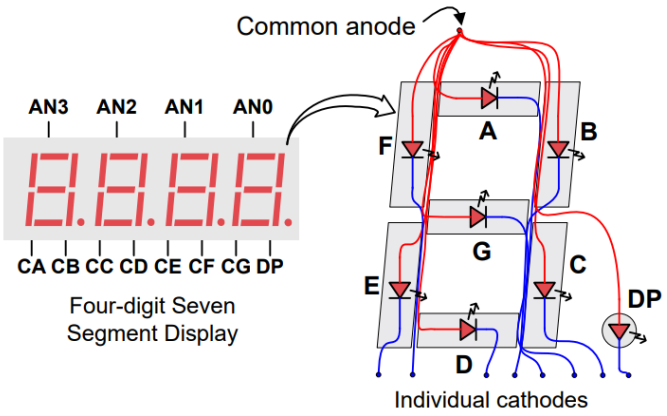


Figure5 : Seven Segment Display Common Anode Circuit Node

RESULTS:

Initially, we executed the test bench for our counter, as depicted in (Figure 6). As intended, the phase_selector cycles through every 5.2 milliseconds, and the seconds increment when a full second elapses. Then we executed thest bench for our main module as "tb_7seg" as decipted in (Figure7). As expected, cathodes select the corresponding digit in the anode phase and the cycle continues in the anode phase.

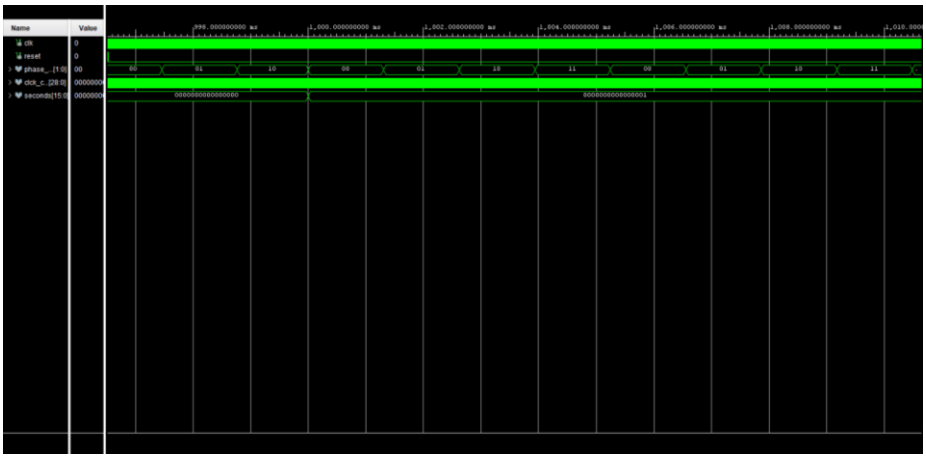


Figure6 : The waveforms of tb_counter

Figure 7 : The waveforms tb_7seg

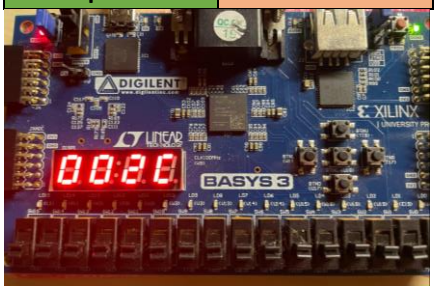
After observing our testbench simulations, we programmed our design into Basys3 and assigned the reset button to the V17 pin, and then we observed some situations. The situations we observed are as shown below.

CASE1:

RESET	1
Past seconds	-
Hex equivalent	-

CASE2:

RESET	0
Past seconds	44
Hex equivalent	2C

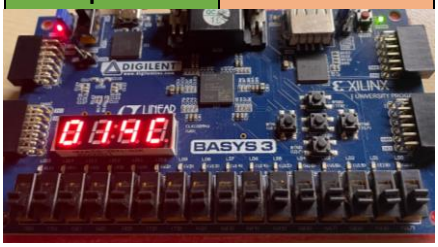


CASE3:

RESET	0
Past seconds	306
Hex equivalent	132

CASE4:

RESET	0
Past seconds	332
Hex equivalent	14C



CONCLUSION:

In this laboratory project, we write a code to display hexadecimal counter on the seven segment display of Basys3. Although the main purpose of the project was to use this displays, I chose hexadecimal counter. The code we wrote was successfully observed on Basys3 and we can see the digits. It was really essential to use “persistence of vision” and clock divider. In order to use these, I did research on what these concepts are and how they are implemented. I encountered a few issues throughout this lab project, namely that the code looked fine but did not work as expected in Basys3. The period of the phase selector was less than 1 ms before being set to 5.2 ms. Therefore, the LEDs could not work properly and meaningless numbers were observed on the screen. With a research and changes we made, this problem was overcome.

APPENDIX:

main_7seg.vhd

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity main_7seg is
Port (
reset : in std_logic ;
clk : in std_logic;
cathode : out std_logic_vector(6 downto 0);
anode :  out std_logic_vector(3 downto 0)
);
end main_7seg;


architecture Behavioral of main_7seg is

component decoder_anode is
Port (
sel_phase : in std_logic_vector(1 downto 0);
decoder_anod : out std_logic_vector(3 downto 0)
);
end component;

component counter_7segment is
Port (
reset : in STD_LOGIC;
clk : in std_logic ;
count_clck : out std_logic_vector (28 downto 0);
sel_phase : out std_logic_vector (1 downto 0);
sec : out std_logic_vector(15 downto 0):= (others => '0')
);
end component;

component LED_driver is
Port (
```

```
num : in std_logic_vector(15 downto 0);
sel_anod: in std_logic_vector(3 downto 0);
sel_cathode : out std_logic_vector(6 downto 0)
);
end component;

signal sec_sig : std_logic_vector(15 downto 0);
signal sel_phase_sig : std_logic_vector(1 downto 0);
signal decoder_anod_sig : std_logic_vector(3 downto 0);

begin

counter_7seg : counter_7segment
Port Map ( reset => reset, clk => clk, sec => sec_sig, sel_phase => sel_phase_sig, count_clck => open);

driver_LED : LED_driver
Port Map (num => sec_sig, sel_anod => decoder_anod_sig, sel_cathode => cathode);

decoder_an : decoder_anode
Port Map( sel_phase => sel_phase_sig , decoder_anod => decoder_anod_sig);

anode <= decoder_anod_sig;

end Behavioral;
```

counter_7seg.vhd

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;
use IEEE.NUMERIC_STD.ALL;

entity counter_7segment is
    Port ( reset : in STD_LOGIC;
           clk : in std_logic ;
           count_clck : out std_logic_vector (28 downto 0);
           sel_phase : out std_logic_vector (1 downto 0);
           sec : out std_logic_vector(15 downto 0):= (others => '0'));
end counter_7segment;
```

```
architecture Behavioral of counter_7segment is

signal clck : std_logic_vector(28 downto 0):= (others => '0');
signal second : std_logic_vector(16 downto 0):= (others => '0');

begin

process (reset,clk)
begin

if reset = '1' then
 clck <= (others => '0');
  second <= (others => '0');
    else
       if rising_edge(clk) then
        clck <= clck + '1';
        if clck = x"5F5E100" then
         second <= second + '1' ;
         clck <= (others => '0') ;
       end if;
    end if;
if second = x"FFFF" then second <= (others => '0');
end if;
end if;
end process;
sel_phase <= clck(18 downto 17);
count_clck <= clck(28 downto 0);
sec <= second(15 downto 0) ;

end Behavioral;
```

driver_LED.vhd

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity LED_driver is
Port (
num : in std_logic_vector(15 downto 0);
```

```
sel_anod: in std_logic_vector(3 downto 0);
sel_cathode : out std_logic_vector(6 downto 0));
end LED_driver;

architecture Behavioral of LED_driver is

signal display_comb: std_logic_vector(3 downto 0);


begin


process(sel_anod) begin
case(sel_anod) is
   when "0111" => display_comb <= num(15 downto 12);
   when "1011" => display_comb <= num(11 downto 8);
   when "1101" => display_comb <= num(7 downto 4);
   when "1110" => display_comb <= num(3 downto 0);
   when others => display_comb <= "0000";
end case;
end process;

process(display_comb) begin
case display_comb is
   when "0000" => sel_cathode <= "0000001";
   when "0001" => sel_cathode <= "1001111";
   when "0010" => sel_cathode <= "0010010";
   when "0011" => sel_cathode <= "0000110";
   when "0100" => sel_cathode <= "1001100";
   when "0101" => sel_cathode <= "0100100";
   when "0110" => sel_cathode <= "0100000";
   when "0111" => sel_cathode <= "0001111";
   when "1000" => sel_cathode <= "0000000";
   when "1001" => sel_cathode <= "0000100";
   when "1010" => sel_cathode <= "0001000";
   when "1011" => sel_cathode <= "1100000";
   when "1100" => sel_cathode <= "0110001";
   when "1101" => sel_cathode <= "1000010";
```

```
   when "1110" => sel_cathode <= "0110000";
   when "1111" => sel_cathode <= "0111000";
   when others => sel_cathode <= "1111111";
end case;
end process;

end Behavioral;
```

decoder_an.vhd

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity decoder_anode is
Port (
sel_phase : in std_logic_vector(1 downto 0);
decoder_anod : out std_logic_vector(3 downto 0));
end decoder_anode;



architecture Behavioral of decoder_anode is

begin

process(sel_phase)
begin
case(sel_phase) is
   when "00" => decoder_anod <= "0111";
   when "01" => decoder_anod <= "1011";
   when "10" => decoder_anod <= "1101";
   when "11" => decoder_anod <= "1110";
   when others => decoder_anod <= "1111";
end case;
end process;
end Behavioral;
```

tb_7seg.vhd

```
library IEEE;
```

```
use IEEE.STD_LOGIC_1164.ALL;

entity tb_7seg is
--  Port ( );
end tb_7seg;

architecture Behavioral of tb_7seg is

signal reset, clk : std_logic;
signal anode: std_logic_vector(3 downto 0);
signal cathode : std_logic_vector(6 downto 0);

begin

D: entity work.main_7seg
Port Map ( clk => clk, reset => reset, anode => anode, cathode => cathode );

clock_process : process
begin
clk <= '0';
wait for 10 ns;
clk <= '1';
wait for 10 ns;

end process;

res_process : process
begin
reset <= '1';
wait for 20 ns;
reset <= '0';
wait;
end process;
end Behavioral;
```

tb_counter.vhd

```
library IEEE;
```

```
use IEEE.STD_LOGIC_1164.ALL;

entity tb_counter is
--  Port ( );
end tb_counter;

architecture Behavioral of tb_counter is
signal clk, reset : std_logic;
signal count_phase: std_logic_vector(1 downto 0);
signal count_clck: std_logic_vector(28 downto 0);
signal count_sec : std_logic_vector(15 downto 0);

begin

dut : entity work.counter_7segment
Port Map (clk => clk, reset => reset, count_clck => count_clck, sel_phase => count_phase, sec => count_sec);

clock_process : process begin
clk <= '0';
wait for 10ns;
clk <= '1';
wait for 10ns;
end process;

reset <= '0';

end Behavioral;
```

cons_7seg.xdc

```
set_property PACKAGE_PIN W7 [get_ports {cathode[6]}]
set_property IOSTANDARD LVCMOS33 [get_ports {cathode[6]}]
set_property PACKAGE_PIN W6 [get_ports {cathode[5]}]
set_property IOSTANDARD LVCMOS33 [get_ports {cathode[5]}]
set_property PACKAGE_PIN U8 [get_ports {cathode[4]}]
set_property IOSTANDARD LVCMOS33 [get_ports {cathode[4]}]
set_property PACKAGE_PIN V8 [get_ports {cathode[3]}]
set_property IOSTANDARD LVCMOS33 [get_ports {cathode[3]}]
```

```
set_property PACKAGE_PIN U5 [get_ports {cathode[2]}]
set_property IOSTANDARD LVCMOS33 [get_ports {cathode[2]}]
set_property PACKAGE_PIN V5 [get_ports {cathode[1]}]
set_property IOSTANDARD LVCMOS33 [get_ports {cathode[1]}]
set_property PACKAGE_PIN U7 [get_ports {cathode[0]}]
set_property IOSTANDARD LVCMOS33 [get_ports {cathode[0]}]

set_property PACKAGE_PIN U2 [get_ports {anode[0]}]
set_property IOSTANDARD LVCMOS33 [get_ports {anode[0]}]
set_property PACKAGE_PIN U4 [get_ports {anode[1]}]
set_property IOSTANDARD LVCMOS33 [get_ports {anode[1]}]
set_property PACKAGE_PIN V4 [get_ports {anode[2]}]
set_property IOSTANDARD LVCMOS33 [get_ports {anode[2]}]
set_property PACKAGE_PIN W4 [get_ports {anode[3]}]
set_property IOSTANDARD LVCMOS33 [get_ports {anode[3]}]

set_property PACKAGE_PIN V17 [get_ports {reset}]
set_property IOSTANDARD LVCMOS33 [get_ports {reset}]

set_property PACKAGE_PIN W5 [get_ports clk]
set_property IOSTANDARD LVCMOS33 [get_ports clk]
```

References:

(Table5)

https://bcisnotes.com/secondsemester/digital-systems/hexadecimal-to-seven-segment/